CURSO DE TI • MÓDULO 5

Implementação de Testes em Projetos Next.js

Jest • React Testing Library

Configuração: Jest + Next.js

1 **Instalação**

```
npm install -D jest jest-environment-jsdom @testing-library/react @testing-library/jest-dom @testing-library/user-event ts-jest
```

2 **jest.config.ts**

```
import type { Config } from 'jest'
const config: Config = {
  testEnvironment: 'jsdom',
  setupFilesAfterFramework: ['<rootDir>/jest.setup.ts'],
  moduleNameMapper: { '^@/(.*)$': '<rootDir>/src/$1' },
  transform: { '^.+\\.(ts|tsx)$': ['ts-jest', {}] },
}
export default config
```

3 **jest.setup.ts**

```
import '@testing-library/jest-dom'
```

4 **package.json scripts**

```
"test": "jest",
"test:watch": "jest --watch",
"test:coverage": "jest --coverage"
```

Use next/jest para configuração otimizada: `const nextJest = require('next/jest')` — já configura paths, assets e SVG

Testes de Integração com React Testing Library

LoginForm.test.tsx

```
import { render, screen, fireEvent }
  from '@testing-library/react'
import userEvent from '@testing-library/user-event'
import { LoginForm } from './LoginForm'

describe('LoginForm', () => {
  it('exibe erro com credenciais inválidas', async () => {
    render(<LoginForm />)

    await userEvent.type(
      screen.getByLabelText('Email'),
      'usuario@teste.com'
    )
    await userEvent.click(
      screen.getByRole('button', { name: /entrar/i })
    )

    expect(await screen.findByText(
      /credenciais inválidas/i
    )).toBeInTheDocument()
  })
})
```

Simula o usuário real

Use getByRole, getByLabelText, getByText — não seletores CSS.

userEvent vs fireEvent

Prefira userEvent: simula cliques, digitação e teclado reais.

Queries por prioridade

Role > Label > Text > TestId. Evite testId sempre que possível.

Async com findBy

Princípio RTL: *"Quanto mais seus testes se assemelham à forma como o software é usado, mais confiança eles fornecem." — Kent C. Dodds*

Cobertura de Código & CI/CD

Configuração de Cobertura (jest.config.ts)

```
collectCoverageFrom: [
  'src/**/*.{ts,tsx}',
  '!src/**/*.stories.tsx',
  '!src/**/*.d.ts',
  '!src/app/layout.tsx',
],
coverageThresholds: {
  global: {
    lines: 80,
    functions: 80,
    branches: 75,
    statements: 80,
  },
},
coverageReporters: ['text', 'html', 'lcov'],
```

80%

Lines mínimo

80%

Functions

75%

Branches

Boas Práticas & Antipadrões

Boas Práticas

- ✓ Siga o padrão AAA: Arrange, Act, Assert
- ✓ Um expect principal por teste (foco claro)
- ✓ Nomes descritivos: 'deve fazer X quando Y'
- ✓ Evite lógica condicional dentro de testes
- ✓ Teste comportamento, não implementação
- ✓ Use factory functions para fixtures reutilizáveis
- ✓ Limpe mocks em afterEach()/beforeEach()
- ✓ Isole efeitos colaterais (DB, APIs, timers)

Antipadrões

- ✗ Testar detalhes internos de implementação
- ✗ Testes dependentes de ordem de execução
- ✗ Snapshots sem manutenção ativa
- ✗ Usar testId excessivamente (prefira role/label)
- ✗ Mock de tudo — torna o teste frágil
- ✗ Ignorar casos de erro e edge cases
- ✗ Testes lentos sem motivo (await desnecessário)
- ✗ Copiar/colar código de teste sem refatorar